DATA STRUCTURES AND ALGORITHMS

Case Study

Unit III - Stacks, Queues and Linked Lists

Implementing Stack and Queue using Linked Lists

Name	Rudrakshi Koul
Roll No.	09
College	Bharati Vidyapeeth's College of Engineering, Lavale

Case Study: Implementing Stack and Queue using Linked Lists

Concept: Dynamic Data Structures

Problem Statement

Array-based stacks and queues have a fixed size and may overflow or waste memory. The goal is to implement both structures dynamically using linked lists.

Stack (LIFO) using a Singly Linked List

- **push(x):** create a node and insert it at the head. O(1)
- **pop():** remove the head node. O(1)
- The head pointer acts as **top**.
- **Applications:** undo in editors, browser back button, infix to postfix conversion, postfix evaluation.

Queue (FIFO) using a Linked List

- Maintain **front** and **rear** pointers.
- **enqueue(x):** insert at the rear. O(1)
- **dequeue():** remove from the front. O(1)
- **Applications:** printer job scheduling, CPU scheduling, customer service lines.

Conclusion: Linked lists make both structures dynamic, with no fixed size and O(1) operations, at the cost of extra memory for pointers.

Python Program

```
# Unit III: Stack and Queue using linked lists
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:                        # LIFO
    def __init__(self):
        self.top = None
    def push(self, x):
        n = Node(x); n.next = self.top; self.top = n
    def pop(self):
        if not self.top:
            return "Underflow"
        x = self.top.data; self.top = self.top.next
        return x

class Queue:                        # FIFO
    def __init__(self):
        self.front = self.rear = None
    def enqueue(self, x):
        n = Node(x)
        if self.rear:
            self.rear.next = n
        else:
```

```
            self.front = n
        self.rear = n
    def dequeue(self):
        if not self.front:
            return "Underflow"
        x = self.front.data; self.front = self.front.next
        if not self.front:
            self.rear = None
        return x

s = Stack()
for x in (10, 20, 30):
    s.push(x)
print("Stack pops:", s.pop(), s.pop(), s.pop(), s.pop())

q = Queue()
for x in ("A", "B", "C"):
    q.enqueue(x)
print("Queue dequeues:", q.dequeue(), q.dequeue(), q.dequeue(), q.dequeue())
```

Output

```
Stack pops: 30 20 10 Underflow
Queue dequeues: A B C Underflow
```